

TAREA

Implementación de un Modelo GRC con Enfoque Tecnológico y SOC Simulado

Objetivo general

Diseñar e implementar un modelo GRC funcional basado en ciberseguridad moderna, integrando:

- Gestión de riesgos
- Seguridad en Cloud y sistemas Linux
- Simulación SOC
- Análisis de logs
- Controles de seguridad
- Indicadores KPI/KRI

Trabajo en equipo

- Grupos de 5 estudiantes
- Duración: 5 días
- Entrega: documento + laboratorio + simulación SOC

Fases del proyecto

Fase 1: Planeación y análisis

- Evaluación de madurez GRC
- Definición de caso de negocio
- Roles (CISO, SOC, DFIR, Cloud Admin)

Fase 2: Diseño

- Alcance del modelo
- Priorización de riesgos
- Arquitectura GRC

Fase 3: Implementación

- Simulación de incidentes
- Gestión de respuesta
- Integración de logs

Fase 4: Herramientas

- SIEM simulado
- Docker / Linux logs
- Scripts de análisis

Fase 5: Monitoreo

- KPIs / KRIs
- Reportes SOC
- Mejora continua

Entregables

- Documento GRC completo
- Laboratorio funcional
- Simulación SOC
- Dashboard de métricas

Indicadores mínimos

- MTTR
- MTTD
- % detección de incidentes
- % logs analizados
- Riesgo residual

2. LABORATORIO PRÁCTICO (DOCKER + LINUX + LOGS REALES)

LABORATORIO: “SOC SIMULADO EN ENTORNO CONTROLADO”

Objetivo del laboratorio

Construir un entorno de seguridad simulado donde los estudiantes puedan:

- Generar logs reales
- Detectar eventos de seguridad
- Analizar actividad sospechosa
- Simular un SOC básico

Arquitectura del laboratorio

- Contenedor Linux (Ubuntu)
- Servicio SSH
- Generador de logs
- Script de análisis
- Simulación de ataques

Paso 1 — Docker (entorno base)

Bash

```
docker run -it ubuntu bash
```

Paso 2 — Instalación de herramientas

Bash

```
apt update  
apt install -y openssh-server net-tools python3
```

Paso 3 — Generación de logs simulados

Bash

```
mkdir /logs  
touch /logs/auth.log
```

Paso 4 — Simulación de intentos de acceso

Bash

```
echo "FAILED login user admin from 192.168.1.100" >> /logs/auth.log  
echo "FAILED login user root from 192.168.1.101" >> /logs/auth.log  
echo "FAILED login user admin from 192.168.1.102" >> /logs/auth.log
```

Paso 5 — Script de análisis (SOC básico)

PYthon

```
with open("/logs/auth.log") as f:  
    logs = f.readlines()  
  
fails = [log for log in logs if "FAILED" in log]  
  
print("Eventos sospechosos detectados:", len(fails))  
  
if len(fails) > 2:  
    print("ALERTA: posible ataque de fuerza bruta")
```

Resultado esperado

- Identificación de ataques simulados
- Análisis de logs
- Detección básica SOC

3. SIMULACIÓN TIPO SOC (DATOS REALES PARA ANÁLISIS)

Objetivo

Simular un SOC real utilizando logs de eventos de seguridad.

Dataset de logs (simulado realista)

```
2026-05-01 10:01 LOGIN_FAIL user=admin ip=185.21.44.10
2026-05-01 10:02 LOGIN_FAIL user=root ip=185.21.44.11
2026-05-01 10:03 LOGIN_FAIL user=admin ip=185.21.44.12
2026-05-01 10:04 LOGIN_SUCCESS user=admin ip=185.21.44.10
2026-05-01 10:05 FILE_ACCESS /etc/passwd user=admin
2026-05-01 10:06 DATA_EXPORT database=clients size=2GB
```

Ejercicio SOC

Detectar:

- Fuerza bruta
- Accesos sospechosos
- Exfiltración de datos
- Escalada de privilegios

Script de análisis SOC

Python

```
logs = open("soc_logs.txt").readlines()

failed = [l for l in logs if "LOGIN_FAIL" in l]
exfiltration = [l for l in logs if "DATA_EXPORT" in l]

print("Fallos de login:", len(failed))
print("Posible exfiltración:", len(exfiltration))

if len(failed) > 2:
    print("ALERTA: ataque de fuerza bruta")

if len(exfiltration) > 0:
    print("ALERTA CRÍTICA: posible fuga de datos")
```

Resultado SOC esperado

- Detección de patrones de ataque
- Identificación de exfiltración
- Simulación de analista SOC

INTEGRACIÓN DE LA TAREA

Relación con GRC

Fase	GRC
Logs	DETECTAR
Análisis	ANALIZAR
Respuesta	RESPONDER
Métricas	MEDIR
Prevención	PRO-ACTUAR

Esta tarea permite que el maestrando:

- Piense como analista SOC real
- Comprenda GRC aplicado a tecnología
- Trabaje con logs reales
- Simule incidentes de ciberseguridad
- Integre teoría + práctica

EXTENSIÓN DE LA TAREA — LABORATORIO AVANZADO GRC + SOC

MÓDULO 1 — SIMULACIÓN CONTROLADA DE RANSOMWARE (LABORATORIO SEGURO)

Objetivo

Simular el comportamiento de un ransomware de forma controlada para:

- Detectar patrones de cifrado masivo
- Identificar comportamiento anómalo en archivos
- Activar respuesta SOC
- Analizar impacto en logs

Escenario del laboratorio

- Carpeta simulada de usuario
- Archivos de prueba
- Script que simula cifrado masivo

Paso 1 — Crear entorno simulado

Bash

```
mkdir /lab_ransom  
cd /lab_ransom  
touch file1.txt file2.txt file3.txt file4.txt
```

Paso 2 — Simulación de “cifrado” (renombrado masivo)

Python

```
import os

folder = "/lab_ransom"

for file in os.listdir(folder):
    new_name = file + ".locked"
    os.rename(os.path.join(folder, file), os.path.join(folder, new_name))

print("Simulación de cifrado completada")
```

Paso 3 — Detección SOC (comportamiento ransomware)

Python

```
import os

folder = "/lab_ransom"
locked_files = [f for f in os.listdir(folder) if ".locked" in f]

print("Archivos afectados:", len(locked_files))

if len(locked_files) > 2:
    print("🚨 ALERTA: posible comportamiento ransomware detectado")
```

Indicadores del módulo

- Número de archivos modificados
- Velocidad de cambios
- Patrón de extensión sospechosa
- Activación de alerta SOC

Relación GRC

- **DETECTAR (D1):** comportamiento anómalo
- **RESPONDER (R1):** activación de contención
- **MEDIR (M2):** impacto del incidente

MÓDULO 2 — INTEGRACIÓN SIEM (WAZUH / SPLUNK SIMULADO)

Objetivo

Simular un entorno SIEM para:

- Centralizar logs

- Detectar eventos de seguridad
- Correlacionar incidentes
- Generar alertas automáticas

Arquitectura SIEM simulada

- Agente de logs (Linux)
- Motor de análisis (Python)
- Dashboard (simulado tipo Wazuh/Splunk)

Paso 1 — Instalación conceptual Wazuh (Docker)

Bash

```
docker run -d --name wazuh wazuh/wazuh
```

Paso 2 — Generación de logs de seguridad

```
2026-05-01 LOGIN_FAIL user=admin ip=10.0.0.5
2026-05-01 LOGIN_FAIL user=root ip=10.0.0.6
2026-05-01 LOGIN_FAIL user=admin ip=10.0.0.7
2026-05-01 FILE_MODIFIED /etc/passwd
2026-05-01 MULTIPLE_ACCESS user=admin
```

Paso 3 — Motor SIEM (correlación de eventos)

Python

```
logs = open("siem_logs.txt").readlines()

failed_logins = [l for l in logs if "LOGIN_FAIL" in l]
file_changes = [l for l in logs if "FILE_MODIFIED" in l]

if len(failed_logins) > 2:
    print("🚨 ALERTA SIEM: posible ataque de fuerza bruta")

if len(file_changes) > 0:
    print("🚨 ALERTA SIEM: modificación crítica de sistema")
```

Paso 4 — Simulación tipo dashboard SOC

Python

```
print("==== SOC DASHBOARD ====")
print("Eventos analizados:", 120)
```

```
print("Alertas críticas:", 3)
print("Incidentes activos:", 1)
print("Estado del sistema: EN RIESGO")
```

Relación con GRC

Componente Relación

DETECTAR	SIEM identifica eventos
RESPONDER	SOC activa acciones
MEDIR	KPIs de seguridad
ANALIZAR	correlación de logs

INDICADORES SIEM

- Número de eventos por segundo
- Tiempo de detección (MTTD)
- Número de alertas críticas
- Precisión de correlación
- Incidentes detectados automáticamente

Con estos dos módulos el maestrando:

- Simula ransomware (sin riesgo real)
- Aprende comportamiento de ataques
- Construye lógica SOC real
- Interpreta SIEM como Wazuh/Splunk
- Integra GRC con DFIR